

Hashmaps:

Saturday, August 8, 2026 1:58 AM

When to use HashMap?

- Agar koi old data yaad rakhna hai.
- Koi bhi sawaal puchenege hashmap se vo exact hona chahiye like - key ki value? or is that this key?, not like - find between/before/after in range.
- Jab kabhi kabhi question me frequency / repetative jaise term use ho.

HashMap in CPP:

- `unordered_map<int i, int j> f;` // (i-> key and j -> value)
- Access using key-> `f[i]`
- to erase a particular key enter -> `f.erase(i)`
- to get size of hashmap i.e. total no. of pairs exists -> `f.size()`
- If we want to check is there in particular key entry present in hashmap or not?
 - IF key is not present-> `if(f.find(i) == f.end())`
 - IF key is present then -> `if(f.find(i) != f.end())`
- IF we want to loop in a hashmap:
 - o `for( auto l : f)`
 - {
 - int key = i.first;
 - int val = i.second;
 - }
- There are 2 type of hashmap in cpp -> **unordered_map (can only have entries with unique key) and unordered_multimap (can have multiple entries with same key).**
- If you want to check weather both hashmaps contain the same charaters with same frequency:
 - o `unordered_map<char, int> freq1` and `unordered_map<char, int> freq2`
`if (freq1 == freq2){ cout << "Frequencies are equal";}`
`else { cout << "Frequencies are different";}`

Question-> LC 387 & LC 242:

- First unique character of string?
- Create a hashmap out of that string where key will be of char type and val will be of int type i.e. its frequency of occurrence throughout the string.
 - o `for(i=0; i<n; i++)`
 - {
 - `f[s[i]]++;`
 - }

Question: Valid Anagram: LC 242:

```
bool isAnagram(string s, string t) {
    bool ans = true;
    int n = s.size();
    int m = t.size();
    unordered_map<char, int> freq1;
    unordered_map<char, int> freq2;
    for(int i=0; i < n ; i++){
        freq1[s[i]]++;
    }
    for(int j=0; j < m ; j++){
        freq2[t[j]]++;
    }
    if(freq1 != freq2){
        ans = false;
    }
    return ans;
}
```

Question: TWO SUM - O(n):

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> seen; // key: number, value: index
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.find(complement) != seen.end()) {
```

```

        return {seen[complement], i};
    }
    seen[nums[i]] = i;
}
return {};
}

```

Question: LC 49: Group Anagrams:

```

vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> groups;
    for (string word : strs) {
        string key = word;
        sort(key.begin(), key.end());
        groups[key].push_back(word);
    }

    vector<vector<string>> result;
    for (auto& pair : groups)
    {
        result.push_back(pair.second);
    }
    return result;
}

```

Question LC 409:

return the longest palindromic size in s

```

int longestPalindrome(string s) {
    int n = s.size();
    int res = 0;
    bool odd = false;
    unordered_map<char, int> f;
    for(int i = 0 ; i < n; i++){
        f[s[i]]++;
    }
    for(auto i : f){
        int val = i.second;
        if(val % 2 == 0){
            res = res + val;
        }
        else{
            odd = true;
        }
    }
    if(odd == false){
        return res;
    }
    else{
        for(auto i : f){
            int valu = i.second;
            if(valu % 2 == 1){
                res = res + (valu - 1);
            }
        }
    }
    return res + 1;
}

```